

Group14_Source_Code1

May 24, 2026

1 Data Gereration

```
[2]: !pip install python-chess pandas tqdm zstandard
```

```
Requirement already satisfied: python-chess in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (1.999)
Requirement already satisfied: pandas in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (1.5.3)
Requirement already satisfied: tqdm in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (4.67.3)
Requirement already satisfied: zstandard in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (0.25.0)
Requirement already satisfied: chess<2,>=1 in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from python-chess) (1.11.2)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: numpy>=1.21.0 in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from pandas) (1.25.2)
Requirement already satisfied: colorama in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from tqdm) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\sachin
kumar\.conda\envs\ml_learning\lib\site-packages (from python-
dateutil>=2.8.1->pandas) (1.17.0)
```

```
[3]: import subprocess
```

```
STOCKFISH_PATH = r"C:\Users\sachin\
↳kumar\Downloads\stockfish-windows-x86-64-avx2\stockfish\stockfish-windows-x86-64-avx2.
↳exe"

engine_process = subprocess.Popen(
    STOCKFISH_PATH,
    universal_newlines=True,
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE
```

```

)

def send(cmd):
    engine_process.stdin.write(cmd + "\n")
    engine_process.stdin.flush()

def read_until(token):
    while True:
        line = engine_process.stdout.readline().strip()
        if token in line:
            break

send("uci")
read_until("uciok")

send("isready")
read_until("readyok")

```

```

[4]: def evaluate_fen(fen, depth=10):
    send(f"position fen {fen}")
    send(f"go depth {depth}")

    eval_cp = 0

    while True:
        line = engine_process.stdout.readline().strip()

        if "score cp" in line:
            eval_cp = int(line.split("score cp")[1].split()[0])

        elif "score mate" in line:
            mate = int(line.split("score mate")[1].split()[0])
            eval_cp = 10000 if mate > 0 else -10000

        elif line.startswith("bestmove"):
            break

    return eval_cp

```

```

[5]: import zstandard as zstd
import io

DATA_PATH = r"C:\ID5030_ML\lichess_db_standard_rated_2017-03.pgn.zst"

# OPEN OUTSIDE (no 'with')
f = open(DATA_PATH, 'rb')
dctx = zstd.ZstdDecompressor()

```

```
stream = dctx.stream_reader(f)
pgn_stream = io.TextIOWrapper(stream, encoding='utf-8')
```

1.1 Importing 20 games data for data visualization

```
[6]: import numpy as np
from tqdm import tqdm
import chess
import chess.pgn

# ----- 12x8x8 PIECE ENCODER -----
piece_map = {
    'P': 0, 'N': 1, 'B': 2, 'R': 3, 'Q': 4, 'K': 5,
    'p': 6, 'n': 7, 'b': 8, 'r': 9, 'q': 10, 'k': 11
}

def fen_to_tensor_12(fen):
    tensor = np.zeros((12, 8, 8), dtype=np.int8)

    board_part = fen.split()[0]
    rows = board_part.split('/')

    for i, row in enumerate(rows):
        col = 0
        for ch in row:
            if ch.isdigit():
                col += int(ch)
            else:
                tensor[piece_map[ch], i, col] = 1
                col += 1

    return tensor

# ----- FEATURE EXTRACTORS -----

def encode_castling(castling):
    return {
        "castle_K": int('K' in castling),
        "castle_Q": int('Q' in castling),
        "castle_k": int('k' in castling),
        "castle_q": int('q' in castling),
    }

def encode_en_passant(ep):
    if ep == '-':
        return -1
```

```

    file = ord(ep[0]) - ord('a')
    rank = int(ep[1]) - 1
    return file + 8 * rank

def material_balance(board):
    values = {
        chess.PAWN: 1, chess.KNIGHT: 3, chess.BISHOP: 3,
        chess.ROOK: 5, chess.QUEEN: 9
    }
    balance = 0
    for piece_type in values:
        balance += len(board.pieces(piece_type, chess.WHITE)) * values[piece_type]
        balance -= len(board.pieces(piece_type, chess.BLACK)) * values[piece_type]
    return balance

def normalize_eval(eval_cp, board):
    return eval_cp if board.turn == chess.WHITE else -eval_cp

def evaluate_fen(fen, depth=10):
    send(f"position fen {fen}")
    send(f"go depth {depth}")

    eval_cp = 0

    while True:
        line = engine_process.stdout.readline().strip()

        if "score cp" in line:
            eval_cp = int(line.split("score cp")[1].split()[0])

        elif "score mate" in line:
            mate = int(line.split("score mate")[1].split()[0])
            eval_cp = 10000 if mate > 0 else -10000

        elif line.startswith("bestmove"):
            break

    return eval_cp

engine_process
send("uci")
read_until("uciok")

send("isready")
read_until("readyok")

```

```

# ----- MAIN -----
rows = []

for game_id in tqdm(range(20)):
    game = chess.pgn.read_game(pgn_stream)
    if game is None:
        break

    board = game.board()

    white_elo = game.headers.get("WhiteElo")
    black_elo = game.headers.get("BlackElo")

    prev_eval = normalize_eval(evaluate_fen(board.fen()), board)
    move_number = 1

    for move in game.mainline_moves():
        player = "white" if board.turn == chess.WHITE else "black"

        fen = board.fen()
        parts = fen.split()

        castling = parts[2]
        en_passant = parts[3]
        halfmove = int(parts[4])
        fullmove = int(parts[5])

        # ---- features BEFORE move ----
        tensor_12 = fen_to_tensor_12(fen)

        castle_dict = encode_castling(castling)
        ep_encoded = encode_en_passant(en_passant)

        is_check = int(board.is_check())
        mobility = board.legal_moves.count()
        mat_balance = material_balance(board)

        piece_moved = board.piece_at(move.from_square)
        piece_type = piece_moved.piece_type if piece_moved else 0

        is_capture = int(board.is_capture(move))
        gives_check = int(board.gives_check(move))

        # ---- push move ----
        board.push(move)

```

```

new_eval = normalize_eval(evaluate_fen(board.fen()), board)
delta = new_eval - prev_eval

row = {
    "game_id": game_id,
    "move_number": move_number,
    "player": player,
    "fen": fen,
    "tensor_12": tensor_12,

    # ---- scalar features ----
    "turn": int(parts[1] == 'w'),
    "en_passant": ep_encoded,
    "halfmove_clock": halfmove,
    "fullmove_number": fullmove,

    "is_check": is_check,
    "mobility": mobility,
    "material_balance": mat_balance,

    "piece_moved": piece_type,
    "is_capture": is_capture,
    "gives_check": gives_check,

    "eval": prev_eval,
    "delta": delta,

    "white_elo": white_elo,
    "black_elo": black_elo
}

# add castling bits
row.update(castle_dict)

rows.append(row)

prev_eval = new_eval
move_number += 1

```

```

100%|
| 20/20 [00:45<00:00, 2.27s/it]

```

```

[7]: import pandas as pd
df = pd.DataFrame(rows)

def classify(delta):

```

```

    if delta <= -300:
        return "blunder"
    elif delta <= -100:
        return "mistake"
    elif delta <= -50:
        return "inaccuracy"
    elif delta < 50:
        return "good"
    else:
        return "excellent"
# ----- labels -----
df["label"] = df["delta"].apply(classify)

# ----- numeric cleanup -----
df["white_elo"] = pd.to_numeric(df["white_elo"], errors="coerce")
df["black_elo"] = pd.to_numeric(df["black_elo"], errors="coerce")

df["elo_diff"] = df["white_elo"] - df["black_elo"]
df["is_white"] = (df["player"] == "white").astype(int)

# ----- drop missing -----
df = df.dropna()

# ----- OUTPUT -----
print(df.head())
print("\nShape:", df.shape)
print("\nLabel distribution:\n", df["label"].value_counts())

```

```

game_id  move_number  player \
0         0           1  white
1         0           2  black
2         0           3  white
3         0           4  black
4         0           5  white

```

```

fen \
0  rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w ...
1  rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR ...
2  rnbqkbnr/ppp1pppp/8/3p4/4P3/8/PPPP1PPP/RNBQKBN...
3  rnbqkbnr/ppp1pppp/8/3P4/8/8/PPPP1PPP/RNBQKBNR ...
4  rnb1kbnr/ppp1pppp/8/3q4/8/8/PPPP1PPP/RNBQKBNR ...

```

```

tensor_12  turn  en_passant \
0  [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,...  1      -1
1  [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,...  0      -1
2  [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,...  1      -1
3  [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,...  0      -1
4  [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,...  1      -1

```

	halfmove_clock	fullmove_number	is_check	...	delta	white_elo	\
0	0	1	0	...	-20	1833	
1	0	1	0	...	32	1833	
2	0	2	0	...	-14	1833	
3	0	2	0	...	11	1833	
4	0	3	0	...	5	1833	

	black_elo	castle_K	castle_Q	castle_k	castle_q	label	elo_diff	\
0	1799	1	1	1	1	good	34	
1	1799	1	1	1	1	good	34	
2	1799	1	1	1	1	good	34	
3	1799	1	1	1	1	good	34	
4	1799	1	1	1	1	good	34	

	is_white
0	1
1	0
2	1
3	0
4	1

[5 rows x 26 columns]

Shape: (1477, 26)

Label distribution:

good	1002
excellent	246
inaccuracy	104
mistake	92
blunder	33

Name: label, dtype: int64

```
[8]: df.iloc[0,0:26]
```

```
[8]: game_id          0
     move_number      1
     player          white
     fen      rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w ...
     tensor_12      [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, ...
     turn          1
     en_passant     -1
     halfmove_clock  0
     fullmove_number  1
     is_check        0
     mobility       20
```


material_balance	0
piece_moved	1
is_capture	0
gives_check	0
eval	69
delta	-20
white_elo	1833
black_elo	1799
castle_K	1
castle_Q	1
castle_k	1
castle_q	1
label	good
elo_diff	34
is_white	1

Name: 0, dtype: object

1.2 Importing 5000 games data for model trainings

```
[10]: # ----- MAIN -----
rows = []

for game_id in tqdm(range(5000)):
    game = chess.pgn.read_game(pgn_stream)
    if game is None:
        break

    board = game.board()

    white_elo = game.headers.get("WhiteElo")
    black_elo = game.headers.get("BlackElo")

    prev_eval = normalize_eval(evaluate_fen(board.fen()), board)
    prev_delta = 0    # initialize

    move_number = 1

    for move in game.mainline_moves():
        fen = board.fen()
        parts = fen.split()

        castling = parts[2]
        en_passant = parts[3]
        halfmove = int(parts[4])

        # ----- tensor (FIXED: no flattening) -----
        tensor = fen_to_tensor_12(fen).astype(np.int8)
```

```

# ----- features -----
castle_dict = encode_castling(castling)
ep_encoded = encode_en_passant(en_passant)

is_check = int(board.is_check())
mobility = board.legal_moves.count()
mat_balance = material_balance(board)

piece = board.piece_at(move.from_square)
piece_type = piece.piece_type if piece else 0

is_capture = int(board.is_capture(move))
gives_check = int(board.gives_check(move))

# ----- ELO -----
w_elo = pd.to_numeric(white_elo, errors="coerce")
b_elo = pd.to_numeric(black_elo, errors="coerce")

elo_diff = w_elo - b_elo
avg_elo = (w_elo + b_elo) / 2

# ----- push move -----
board.push(move)

new_eval = normalize_eval(evaluate_fen(board.fen()), board)
delta = new_eval - prev_eval

# ----- row -----
row = {
    "move_number": move_number,
    "turn": int(parts[1] == 'w'),

    "en_passant": ep_encoded,
    "halfmove_clock": halfmove,

    "is_check": is_check,
    "mobility": mobility,
    "material_balance": mat_balance,

    "piece_moved": piece_type,
    "is_capture": is_capture,
    "gives_check": gives_check,

    "prev_eval": prev_eval,
    "prev_delta": prev_delta,

```

```

        "eval": prev_eval,
        "delta": delta,

        "elo_diff": elo_diff,
        "avg_elo": avg_elo,

        # FIX: store tensor as single column
        "tensor_12": tensor
    }

    # add castling features
    row.update(castle_dict)

    rows.append(row)

    # ----- update for next move -----
    prev_delta = delta
    prev_eval = new_eval

    move_number += 1

```

```

100%|
| 5000/5000 [9:31:29<00:00, 6.86s/it]

```

```

[11]: df = pd.DataFrame(rows)

# ----- label -----
df["label"] = df["delta"].apply(classify)

# ----- clean -----
df = df.dropna()

# ----- DEFINE X, y -----
X = df.drop(columns=["label"])
y = df["label"]

# ----- OUTPUT -----
print("X shape:", X.shape)
print("y shape:", y.shape)
print(df.head())

```

```
X shape: (337981, 21)
```

```
y shape: (337981,)
```

	move_number	turn	en_passant	halfmove_clock	is_check	mobility	\
0	1	1	-1	0	0	20	
1	2	0	-1	0	0	20	
2	3	1	-1	0	0	30	
3	4	0	-1	0	0	21	

4	5	1	-1	1	0	38
---	---	---	----	---	---	----

	material_balance	piece_moved	is_capture	gives_check	...	eval	delta	\
0	0	1	0	0	...	38	0	
1	0	1	0	0	...	38	25	
2	0	1	0	0	...	63	-2	
3	0	3	0	0	...	61	7	
4	0	3	0	0	...	68	-52	

	elo_diff	avg_elo	tensor_12																\
0	250	1577.0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...																
1	250	1577.0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...																
2	250	1577.0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...																
3	250	1577.0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...																
4	250	1577.0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...																

	castle_K	castle_Q	castle_k	castle_q	label
0	1	1	1	1	good
1	1	1	1	1	good
2	1	1	1	1	good
3	1	1	1	1	good
4	1	1	1	1	inaccuracy

[5 rows x 22 columns]

```
[12]: X.iloc[0,0:22]
```

```
[12]: move_number          1
      turn                1
      en_passant          -1
      halfmove_clock       0
      is_check             0
      mobility            20
      material_balance      0
      piece_moved          1
      is_capture           0
      gives_check          0
      prev_eval            38
      prev_delta           0
      eval                38
      delta                0
      elo_diff             250
      avg_elo              1577.0
      tensor_12            [[[0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0], ...
      castle_K              1
      castle_Q              1
      castle_k              1
```

castle_q
Name: 0, dtype: object

1

```
[14]: !pip install pyarrow
```

Collecting pyarrow

Downloading pyarrow-24.0.0-cp310-cp310-win_amd64.whl.metadata (3.0 kB)

Downloading pyarrow-24.0.0-cp310-cp310-win_amd64.whl (27.3 MB)

```
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.0/27.3 MB ? eta -:--:--
----- 0.3/27.3 MB ? eta -:--:--
----- 0.5/27.3 MB 837.5 kB/s eta 0:00:32
----- 0.5/27.3 MB 837.5 kB/s eta 0:00:32
- ----- 0.8/27.3 MB 713.3 kB/s eta 0:00:38
- ----- 0.8/27.3 MB 713.3 kB/s eta 0:00:38
- ----- 1.0/27.3 MB 751.1 kB/s eta 0:00:35
- ----- 1.3/27.3 MB 808.5 kB/s eta 0:00:33
-- ----- 1.6/27.3 MB 806.6 kB/s eta 0:00:32
-- ----- 1.6/27.3 MB 806.6 kB/s eta 0:00:32
-- ----- 1.8/27.3 MB 774.4 kB/s eta 0:00:33
-- ----- 1.8/27.3 MB 774.4 kB/s eta 0:00:33
--- ----- 2.1/27.3 MB 743.2 kB/s eta 0:00:34
--- ----- 2.1/27.3 MB 743.2 kB/s eta 0:00:34
--- ----- 2.4/27.3 MB 729.4 kB/s eta 0:00:35
--- ----- 2.4/27.3 MB 729.4 kB/s eta 0:00:35
--- ----- 2.9/27.3 MB 773.1 kB/s eta 0:00:32
--- ----- 3.1/27.3 MB 816.6 kB/s eta 0:00:30
--- ----- 3.4/27.3 MB 864.0 kB/s eta 0:00:28
----- 3.7/27.3 MB 882.9 kB/s eta 0:00:27
----- 3.9/27.3 MB 866.6 kB/s eta 0:00:27
----- 4.2/27.3 MB 889.2 kB/s eta 0:00:26
----- 4.5/27.3 MB 913.0 kB/s eta 0:00:25
----- 4.7/27.3 MB 935.0 kB/s eta 0:00:25
----- 5.0/27.3 MB 946.6 kB/s eta 0:00:24
----- 5.0/27.3 MB 946.6 kB/s eta 0:00:24
----- 5.2/27.3 MB 921.3 kB/s eta 0:00:24
----- 5.5/27.3 MB 921.7 kB/s eta 0:00:24
----- 5.5/27.3 MB 921.7 kB/s eta 0:00:24
----- 5.8/27.3 MB 910.4 kB/s eta 0:00:24
----- 6.3/27.3 MB 943.4 kB/s eta 0:00:23
----- 6.3/27.3 MB 943.4 kB/s eta 0:00:23
----- 6.3/27.3 MB 943.4 kB/s eta 0:00:23
```

```

----- 6.6/27.3 MB 894.8 kB/s eta 0:00:24
----- 6.8/27.3 MB 909.8 kB/s eta 0:00:23
----- 7.1/27.3 MB 922.2 kB/s eta 0:00:22
----- 7.1/27.3 MB 922.2 kB/s eta 0:00:22
----- 7.1/27.3 MB 922.2 kB/s eta 0:00:22
----- 7.3/27.3 MB 872.7 kB/s eta 0:00:23
----- 7.3/27.3 MB 872.7 kB/s eta 0:00:23
----- 7.3/27.3 MB 872.7 kB/s eta 0:00:23
----- 7.6/27.3 MB 852.5 kB/s eta 0:00:24
----- 7.9/27.3 MB 864.2 kB/s eta 0:00:23
----- 7.9/27.3 MB 864.2 kB/s eta 0:00:23
----- 8.1/27.3 MB 837.4 kB/s eta 0:00:23
----- 8.1/27.3 MB 837.4 kB/s eta 0:00:23
----- 8.4/27.3 MB 828.1 kB/s eta 0:00:23
----- 8.4/27.3 MB 828.1 kB/s eta 0:00:23
----- 8.9/27.3 MB 842.7 kB/s eta 0:00:22
----- 9.2/27.3 MB 847.6 kB/s eta 0:00:22
----- 9.4/27.3 MB 857.2 kB/s eta 0:00:21
----- 9.7/27.3 MB 870.3 kB/s eta 0:00:21
----- 10.0/27.3 MB 884.2 kB/s eta 0:00:20
----- 10.2/27.3 MB 887.9 kB/s eta 0:00:20
----- 10.5/27.3 MB 885.4 kB/s eta 0:00:19
----- 10.5/27.3 MB 885.4 kB/s eta 0:00:19
----- 10.5/27.3 MB 885.4 kB/s eta 0:00:19
----- 10.5/27.3 MB 885.4 kB/s eta 0:00:19
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 10.7/27.3 MB 856.0 kB/s eta 0:00:20
----- 11.0/27.3 MB 783.4 kB/s eta 0:00:21
----- 11.0/27.3 MB 783.4 kB/s eta 0:00:21
----- 11.0/27.3 MB 783.4 kB/s eta 0:00:21
----- 11.3/27.3 MB 764.3 kB/s eta 0:00:21
----- 11.3/27.3 MB 764.3 kB/s eta 0:00:21
----- 11.5/27.3 MB 761.8 kB/s eta 0:00:21
----- 11.5/27.3 MB 761.8 kB/s eta 0:00:21
----- 11.5/27.3 MB 761.8 kB/s eta 0:00:21
----- 11.8/27.3 MB 752.5 kB/s eta 0:00:21
----- 11.8/27.3 MB 752.5 kB/s eta 0:00:21
----- 11.8/27.3 MB 752.5 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21
----- 12.1/27.3 MB 733.0 kB/s eta 0:00:21

```

[illegible]

```

----- 12.8/27.3 MB 456.5 kB/s eta 0:00:32
----- 12.8/27.3 MB 456.5 kB/s eta 0:00:32
----- 13.1/27.3 MB 458.5 kB/s eta 0:00:31
----- 13.4/27.3 MB 462.4 kB/s eta 0:00:31
----- 13.6/27.3 MB 468.6 kB/s eta 0:00:30
----- 13.9/27.3 MB 474.6 kB/s eta 0:00:29
----- 14.4/27.3 MB 488.1 kB/s eta 0:00:27
----- 14.7/27.3 MB 493.2 kB/s eta 0:00:26
----- 14.9/27.3 MB 499.5 kB/s eta 0:00:25
----- 15.2/27.3 MB 503.3 kB/s eta 0:00:24
----- 15.2/27.3 MB 503.3 kB/s eta 0:00:24
----- 15.5/27.3 MB 503.1 kB/s eta 0:00:24
----- 15.7/27.3 MB 506.8 kB/s eta 0:00:23
----- 15.7/27.3 MB 506.8 kB/s eta 0:00:23
----- 15.7/27.3 MB 506.8 kB/s eta 0:00:23
----- 16.0/27.3 MB 501.2 kB/s eta 0:00:23
----- 16.3/27.3 MB 501.5 kB/s eta 0:00:22
----- 16.5/27.3 MB 507.6 kB/s eta 0:00:22
----- 16.8/27.3 MB 508.9 kB/s eta 0:00:21
----- 16.8/27.3 MB 508.9 kB/s eta 0:00:21
----- 16.8/27.3 MB 508.9 kB/s eta 0:00:21
----- 17.0/27.3 MB 502.0 kB/s eta 0:00:21
----- 17.0/27.3 MB 502.0 kB/s eta 0:00:21
----- 17.3/27.3 MB 500.4 kB/s eta 0:00:20
----- 17.6/27.3 MB 503.1 kB/s eta 0:00:20
----- 17.8/27.3 MB 508.7 kB/s eta 0:00:19
----- 18.1/27.3 MB 508.1 kB/s eta 0:00:19
----- 18.4/27.3 MB 499.1 kB/s eta 0:00:18
----- 18.6/27.3 MB 499.9 kB/s eta 0:00:18
----- 18.6/27.3 MB 499.9 kB/s eta 0:00:18
----- 18.6/27.3 MB 499.9 kB/s eta 0:00:18
----- 18.9/27.3 MB 490.9 kB/s eta 0:00:18
----- 18.9/27.3 MB 490.9 kB/s eta 0:00:18
----- 19.1/27.3 MB 474.8 kB/s eta 0:00:18
----- 19.1/27.3 MB 474.8 kB/s eta 0:00:18
----- 19.4/27.3 MB 474.8 kB/s eta 0:00:17
----- 19.7/27.3 MB 475.3 kB/s eta 0:00:16
----- 19.9/27.3 MB 476.6 kB/s eta 0:00:16
----- 20.2/27.3 MB 482.3 kB/s eta 0:00:15
----- 20.4/27.3 MB 481.4 kB/s eta 0:00:15
----- 20.7/27.3 MB 481.1 kB/s eta 0:00:14
----- 21.0/27.3 MB 488.2 kB/s eta 0:00:13
----- 21.0/27.3 MB 488.2 kB/s eta 0:00:13
----- 21.2/27.3 MB 490.6 kB/s eta 0:00:13
----- 21.5/27.3 MB 489.6 kB/s eta 0:00:12
----- 22.0/27.3 MB 499.2 kB/s eta 0:00:11
----- 22.3/27.3 MB 504.1 kB/s eta 0:00:10
----- 22.3/27.3 MB 504.1 kB/s eta 0:00:10

```



```

----- 22.3/27.3 MB 504.1 kB/s eta 0:00:10
----- 22.5/27.3 MB 504.4 kB/s eta 0:00:10
----- 22.8/27.3 MB 509.2 kB/s eta 0:00:09
----- 23.1/27.3 MB 509.2 kB/s eta 0:00:09
----- 23.3/27.3 MB 515.7 kB/s eta 0:00:08
----- 23.3/27.3 MB 515.7 kB/s eta 0:00:08
----- 23.3/27.3 MB 515.7 kB/s eta 0:00:08
----- 23.3/27.3 MB 515.7 kB/s eta 0:00:08
----- 23.6/27.3 MB 507.1 kB/s eta 0:00:08
----- 23.9/27.3 MB 500.2 kB/s eta 0:00:07
----- 23.9/27.3 MB 500.2 kB/s eta 0:00:07
----- 24.1/27.3 MB 499.6 kB/s eta 0:00:07
----- 24.1/27.3 MB 499.6 kB/s eta 0:00:07
----- 24.4/27.3 MB 475.3 kB/s eta 0:00:07
----- 24.4/27.3 MB 475.3 kB/s eta 0:00:07
----- 24.6/27.3 MB 475.1 kB/s eta 0:00:06
----- 24.6/27.3 MB 475.1 kB/s eta 0:00:06
----- 24.9/27.3 MB 481.4 kB/s eta 0:00:05
----- 25.2/27.3 MB 484.9 kB/s eta 0:00:05
----- 25.2/27.3 MB 484.9 kB/s eta 0:00:05
----- 25.2/27.3 MB 484.9 kB/s eta 0:00:05
----- 25.4/27.3 MB 499.3 kB/s eta 0:00:04
----- 25.4/27.3 MB 499.3 kB/s eta 0:00:04
----- 25.7/27.3 MB 498.4 kB/s eta 0:00:04
----- 26.0/27.3 MB 504.1 kB/s eta 0:00:03
----- 26.0/27.3 MB 504.1 kB/s eta 0:00:03
----- 26.0/27.3 MB 504.1 kB/s eta 0:00:03
----- 26.2/27.3 MB 506.5 kB/s eta 0:00:03
----- 26.2/27.3 MB 506.5 kB/s eta 0:00:03
----- 26.2/27.3 MB 506.5 kB/s eta 0:00:03
----- 26.5/27.3 MB 500.9 kB/s eta 0:00:02
----- 26.5/27.3 MB 500.9 kB/s eta 0:00:02
----- 26.7/27.3 MB 501.7 kB/s eta 0:00:02
----- 26.7/27.3 MB 501.7 kB/s eta 0:00:02
----- 27.0/27.3 MB 509.8 kB/s eta 0:00:01
----- 27.0/27.3 MB 509.8 kB/s eta 0:00:01
----- 27.0/27.3 MB 509.8 kB/s eta 0:00:01
----- 27.3/27.3 MB 506.4 kB/s 0:00:47

```

Installing collected packages: pyarrow

Successfully installed pyarrow-24.0.0

Saving scalar data

```
[16]: df_scalar = df.drop(columns=["tensor_12"])

df_scalar.to_parquet("datasets_chess/chess_scalar.parquet", index=False)
```

Saving tensor data

```
[17]: import numpy as np

X_tensor = np.stack(df["tensor_12"].values)

np.save("datasets_chess/chess_tensor.npy", X_tensor)
```